Java OOP Concepts — Complete Guide

Topics Covered:

- Object Creation (`new` keyword)
- Constructors (Default, Parameterized, Overloading)
- `this` Keyword
- Static Members (Variables, Methods, Blocks)

---

1. Object Creation: `new` Keyword

What is the `new` Keyword?

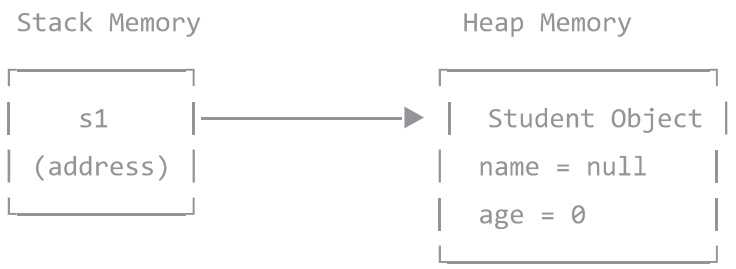
The `new` keyword creates an object from a class. It allocates memory and calls the constructor.

```
Student s1 = new Student();
```

Step-by-Step Breakdown

Step	Code	What Happens
1	`Student`	Class name (type of object)
2	`s1`	Reference variable (stores address)
3	`new`	Allocates memory in Heap
4	`Student()`	Calls constructor to initialize

Memory Model



- **Stack**: Stores reference variables (fast, small)
- **Heap**: Stores actual objects (large)

Complete Example

```
class Student {
    String name;
    int age;
}

public class Main {
    public static void main(String[] args) {
        // Step 1: Create reference only
        Student s1;           // s1 = null

        // Step 2: Create object and assign
        s1 = new Student();   // s1 points to object

        // Or in one line:
        Student s2 = new Student();

        // Step 3: Access variables
        s1.name = "Aslam";
        s1.age = 20;

        System.out.println(s1.name);  // Aslam
        System.out.println(s1.age);   // 20
    }
}
```

Multiple References to Same Object

```
Student s1 = new Student();
s1.name = "Aslam";

Student s2 = s1;    // s2 points to SAME object

s2.name = "Rahim";  // Changes shared object

System.out.println(s1.name);  // Rahim (s1 affected too!)
```

Null Reference

```
Student s1 = null;        // Points to nothing
s1.name = "Aslam";        // ❌ NullPointerException!

// Solution: Check first
if (s1 != null) {
    s1.name = "Aslam";
}
```

---

2. Constructors

What is a Constructor?

A constructor is a **special method** that:
- Runs **automatically** when object is created
- Has **same name** as the class
- Has **no return type** (not even void)
- **Initializes** the object

2.1 Default Constructor

A constructor with **no parameters**.

Implicit (Java Provides Automatically)

```
class Student {
    String name;
    int age;
    boolean isActive;
    // Java provides: Student() { }
}

Student s1 = new Student();
// name = null, age = 0, isActive = false
```

Default Values Table

Data Type	Default Value
int, byte, short, long	0
float, double	0.0
boolean	false
char	'\u0000'
Object (String, etc.)	null

Explicit Default Constructor

```
class Student {
    String name;
    int age;

    Student() {
        System.out.println("Student created");
        name = "Unknown";
        age = 0;
    }
}
```

2.2 Parameterized Constructor

Accepts **arguments** to initialize with specific values.

```
class Student {
    String name;
    int age;
    String school;

    Student(String studentName, int studentAge, String studentSchool) {
        name = studentName;
        age = studentAge;
        school = studentSchool;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("School: " + school);
    }
}

// Usage
Student s1 = new Student("Aslam", 20, "Dhaka College");
s1.display();
```

Important Rule

If you define ANY constructor, Java doesn't provide default:

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

Student s1 = new Student("Aslam");  // ✅ Works
Student s2 = new Student();         // ❌ Error!
```

2.3 Constructor Overloading

Multiple constructors with **different parameters** in the same class.

```
class Student {
    String name;
    int age;
    String course;
    double gpa;

    // Constructor 1: No parameters
    Student() {
        name = "Unknown";
        age = 0;
        course = "Not Assigned";
        gpa = 0.0;
    }

    // Constructor 2: Only name
    Student(String name) {
        this.name = name;
        this.age = 0;
        this.course = "Not Assigned";
        this.gpa = 0.0;
    }

    // Constructor 3: Name and age
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.course = "Not Assigned";
        this.gpa = 0.0;
    }

    // Constructor 4: All parameters
    Student(String name, int age, String course, double gpa) {
        this.name = name;
        this.age = age;
        this.course = course;
        this.gpa = gpa;
    }
}

// Multiple ways to create objects:
```

```
Student s1 = new Student();
Student s2 = new Student("Aslam");
Student s3 = new Student("Rahim", 22);
Student s4 = new Student("Karim", 21, "CSE", 3.75);
```

How Java Matches Constructors

Call	Parameters	Constructor Used
`new Student()`	0	Constructor 1
`new Student("Aslam")`	1 String	Constructor 2
`new Student("Rahim", 22)`	String + int	Constructor 3
`new Student("Karim", 21, "CSE", 3.75)`	4 params	Constructor 4

---

3. `this` Keyword

What is `this` ?

`this` is a reference to the **current object** — the object on which the method is called.

3.1 Distinguish Instance Variable from Parameter

Problem:

```
Student(String name, int age) {
    name = name;   // ❌ Parameter assigns to itself
    age = age;     // Instance variable stays null/0
}
```

Solution:

```
Student(String name, int age) {
    this.name = name;   // ✅ this.name = instance variable
    this.age = age;     // name = parameter
}
```

Visual:

```
this.name = name;
  ↓         ↓
  |         └── Parameter (passed in)
  └── Object's variable (where to store)
```

3.2 Call Another Constructor (Constructor Chaining)

`this()` calls another constructor in the same class.

```
class Student {
    String name;
    int age;
    String course;

    Student() {
        this("Unknown", 0, "Not Assigned");  // Calls Constructor 3
    }

    Student(String name) {
        this(name, 0, "Not Assigned");       // Calls Constructor 3
    }

    Student(String name, int age, String course) {
        this.name = name;
        this.age = age;
        this.course = course;
    }
}
```

Rule: `this()` must be the **first statement**:

```
// ❌ Wrong
Student(String name) {
    System.out.println("Hi");  // Error!
    this(name, 0);
}

// ✅ Correct
Student(String name) {
    this(name, 0);             // First line
    System.out.println("Hi");
}
```

3.3 Return Current Object (Method Chaining)

`return this;` enables method chaining.

```
class Calculator {
    int result;

    Calculator() {
        result = 0;
    }

    Calculator add(int value) {
        result += value;
        return this;
    }

    Calculator subtract(int value) {
        result -= value;
        return this;
    }

    Calculator multiply(int value) {
        result *= value;
        return this;
    }

    void showResult() {
        System.out.println("Result: " + result);
    }
}

// Method chaining:
Calculator calc = new Calculator();
calc.add(10).multiply(5).subtract(20).showResult();  // Result: 30
```

How it works:

```
calc.add(10)      → returns calc
    .multiply(5)  → returns calc
    .subtract(20) → returns calc
    .showResult() → prints result
```

3.4 Pass Current Object as Argument

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }

    void enrollIn(Course course) {
        course.addStudent(this);  // Pass current object
    }
}

class Course {
    String courseName;

    void addStudent(Student student) {
        System.out.println(student.name + " enrolled in " + courseName);
    }
}
```

---

4. Static Members

What is Static?

`static` means belonging to the **class**, not any specific object.

Analogy:

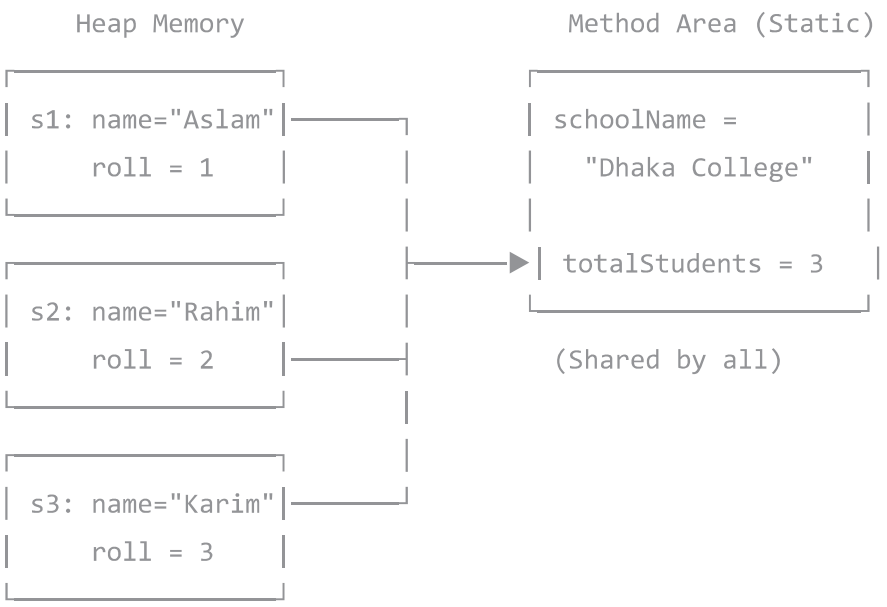
- Each student has their own name → Instance variable
- School name is same for all → Static variable

4.1 Static Variables

One copy shared by ALL objects.

```
class Student {
    // Instance variables (each object has own copy)
    String name;
    int rollNumber;

    // Static variables (ONE copy for all)
    static String schoolName = "Dhaka College";
    static int totalStudents = 0;

    Student(String name) {
        this.name = name;
        totalStudents++;
        this.rollNumber = totalStudents;
    }

    void display() {
        System.out.println("Roll: " + rollNumber +
                           " | Name: " + name +
                           " | School: " + schoolName);
    }
}

// Usage
Student s1 = new Student("Aslam");
Student s2 = new Student("Rahim");
Student s3 = new Student("Karim");

s1.display();  // Roll: 1 | Name: Aslam | School: Dhaka College
s2.display();  // Roll: 2 | Name: Rahim | School: Dhaka College
s3.display();  // Roll: 3 | Name: Karim | School: Dhaka College

System.out.println("Total: " + Student.totalStudents);  // 3

// Change for ALL:
Student.schoolName = "Rajshahi College";
```

Memory Model:



4.2 Static Methods

Called using **class name**, no object needed.

```
class MathHelper {

    static int add(int a, int b) {
        return a + b;
    }

    static int multiply(int a, int b) {
        return a * b;
    }

    static double circleArea(double radius) {
        return 3.1416 * radius * radius;
    }

    static boolean isEven(int number) {
        return number % 2 == 0;
    }
}

// Call without object:
int sum = MathHelper.add(10, 20);           // 30
int product = MathHelper.multiply(5, 7);    // 35
double area = MathHelper.circleArea(5.0);   // 78.54
boolean even = MathHelper.isEven(10);       // true
```

4.3 Static vs Instance — Access Rules

```
class Demo {
    int instanceVar = 10;
    static int staticVar = 20;

    void instanceMethod() {
        System.out.println(instanceVar);   // ✅
        System.out.println(staticVar);     // ✅
        staticMethod();                    // ✅
    }

    static void staticMethod() {
        // System.out.println(instanceVar); // ❌
        System.out.println(staticVar);      // ✅
        // instanceMethod();                // ❌
        // this.staticVar;                  // ❌
    }
}
```

Action	Instance Method	Static Method
Access instance variable	✅ Yes	❌ No
Access static variable	✅ Yes	✅ Yes
Call instance method	✅ Yes	❌ No
Call static method	✅ Yes	✅ Yes
Use `this`	✅ Yes	❌ No

Why? Static belongs to class, not any object. It doesn't know which object's variables to access.

4.4 Static Block

Runs **once** when class first loads.

```
class Database {
    static String url;
    static String username;

    static {
        System.out.println("Loading database settings...");
        url = "jdbc:mysql://localhost:3306/mydb";
        username = "admin";
        System.out.println("Database ready!");
    }

    static void connect() {
        System.out.println("Connecting to: " + url);
    }
}

// First call - static block runs
Database.connect();
// Output:
// Loading database settings...
// Database ready!
// Connecting to: jdbc:mysql://localhost:3306/mydb

// Second call - static block does NOT run again
Database.connect();
// Output:
// Connecting to: jdbc:mysql://localhost:3306/mydb
```

4.5 Static Final (Constants)

Values that **never change**.

```
class Constants {
    static final double PI = 3.14159265359;
    static final int MAX_STUDENTS = 100;
    static final String APP_NAME = "MyApp";
}

// Usage:
double area = Constants.PI * radius * radius;

// Constants.PI = 3.14;  // ❌ Error! Cannot modify final
```

---

5. Complete Example

```
class BankAccount {
    // Instance variables
    private String accountHolder;
    private String accountNumber;
    private double balance;

    // Static variables
    private static String bankName = "National Bank";
    private static int totalAccounts = 0;
    private static double interestRate = 0.05;

    // Static block
    static {
        System.out.println("Banking System Starting");
        System.out.println("Bank: " + bankName);
    }

    // Default constructor
    BankAccount() {
        this("Unknown", 0);
    }

    // Parameterized constructor
    BankAccount(String holder, double initialDeposit) {
        this.accountHolder = holder;
        this.balance = initialDeposit;
        this.accountNumber = "ACC" + (1000 + ++totalAccounts);
    }

    // Method chaining with this
    BankAccount deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
        }
        return this;
    }

    BankAccount withdraw(double amount) {
```

```
        if (amount > 0 && amount <= this.balance) {
            this.balance -= amount;
        }
        return this;
    }

    void applyInterest() {
        this.balance += this.balance * interestRate;
    }

    // Static methods
    static void setInterestRate(double rate) {
        interestRate = rate;
    }

    static int getTotalAccounts() {
        return totalAccounts;
    }

    void showDetails() {
        System.out.println("Account: " + accountNumber);
        System.out.println("Holder: " + accountHolder);
        System.out.println("Balance: $" + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        // Create accounts
        BankAccount acc1 = new BankAccount("Aslam", 50000);
        BankAccount acc2 = new BankAccount("Rahim", 30000);

        // Method chaining
        acc1.deposit(10000).withdraw(5000).deposit(2000);

        // Apply interest
        acc1.applyInterest();

        // Display
        acc1.showDetails();
```

```
        // Static method
        System.out.println("Total: " + BankAccount.getTotalAccounts());

        // Change rate for all
        BankAccount.setInterestRate(0.07);
    }
}
```

Quick Reference

Object Creation

```
ClassName obj = new ClassName();
```

Constructors

```
// Default
ClassName() { }

// Parameterized
ClassName(Type param) {
    this.field = param;
}

// Overloading
ClassName() { }
ClassName(String s) { }
ClassName(String s, int n) { }
```

this Keyword

```
this.variable = parameter;   // Distinguish
this(args);                  // Call constructor
return this;                 // Method chaining
```

Static Members

```
static int count;            // Variable
static void method() { }     // Method
static { }                   // Block
static final double PI = 3.14; // Constant
```

Summary Table

Concept	Description
`new` keyword	Allocates memory, calls constructor
Default Constructor	No parameters, auto-provided
Parameterized Constructor	Accepts arguments
Constructor Overloading	Multiple constructors, different params
`this` keyword	Reference to current object
`this()`	Call another constructor
Static Variable	Shared by all objects
Static Method	Called via class name
Static Block	Runs once on class load
`static final`	Constant value

Created for: Md. Aslam Howlader

Date: February 2026

'; triggerDownload(docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html;charset=utf-8'); } function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); // Force light mode for PDF (dark backgrounds print poorly) const wasDark = document.documentElement.classList.contains('dark'); if (wasDark) document.documentElement.classList.remove('dark'); const element = document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename: docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale: 1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' }, pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if (wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme: dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !== 'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i < mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const container = document.createElement('div'); container.className = 'mermaid-container'; container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark; document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */ window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark); updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) {} } });